

Slides 程序分析完整模拟卷 09（题目与完整解析）

卷面说明

考试时间：120 分钟

总分：100 分

说明：本卷独立覆盖流敏感、路径敏感、上下文敏感、符号执行、SAT、CNF、Tseitin 和可满足性等价证明。

A 卷题目

一、三种敏感性与符号状态（20 分）

给定程序：

```
x = 0;
if (c)
    x = 1;
y = x;

int id(int p) { return p; }
a = id(1);    // call site C1
b = id(2);    // call site C2
```

1. 分别定义 flow-sensitive、path-sensitive、context-sensitive。
2. 流不敏感分析把 $x=0$ 和 $x=1$ 合并后，在 $y=x$ 处得到什么集合。
3. 流敏感但路径不敏感分析在 $y=x$ 处得到什么集合。
4. 路径敏感分析在 $y=x$ 处保留哪两组“路径条件 + x 值”。
5. 上下文不敏感和上下文敏感分析 `id` 时， a 、 b 的返回值结果分别是什么。

二、符号执行路径枚举（20 分）

程序：

```
if (x > 0)
    y = x + 1;
else
    y = 1 - x;

if (y == 2)
    error();
```

1. 写出第一条路径的路径条件和符号状态。
2. 写出第二条路径的路径条件和符号状态。
3. 判断 `error()` 是否可达。
4. 给出能到达 `error()` 的输入。
5. 说明符号执行为什么可能发生路径爆炸。

三、SAT、Validity 与 CNF（20 分）

回答下列问题：

1. 什么是 satisfiable。
2. 什么是 valid。
3. satisfiability 和 validity 的对偶关系是什么。
4. 什么是 CNF。
5. 判断下列表达式是否为 CNF，并说明原因：

```
(a or b) and (not c or d)
not (a and b)
(a or (b and c))
```

四、Tseitin Transformation（20 分）

对公式：

$$F = (a \text{ or } b) \text{ and not } (c \text{ and } d)$$

1. 为子公式引入新变量。 2. 写出每个新变量与子公式等价的约束。 3. 把约束转换成 CNF 子句。 4. 写出最终的 equisatisfiable CNF。 5. 说明 Tseitin 转换为什么比直接展开更适合 SAT 求解。

五、Equisatisfiability 证明 (20 分)

证明下列两个公式 equisatisfiable:

$$F1 = (a_0 \text{ or } a_1 \text{ or } \dots \text{ or } a_n \text{ or } c) \text{ and } (b_0 \text{ or } b_1 \text{ or } \dots \text{ or } b_m \text{ or not } c)$$

$$F2 = (a_0 \text{ or } a_1 \text{ or } \dots \text{ or } a_n \text{ or } b_0 \text{ or } b_1 \text{ or } \dots \text{ or } b_m)$$

1. 设 $A = a_0 \text{ or } a_1 \text{ or } \dots \text{ or } a_n$, $B = b_0 \text{ or } b_1 \text{ or } \dots \text{ or } b_m$, 重写两个公式。 2. 证明若 $F1$ 可满足, 则 $F2$ 可满足。 3. 证明若 $F2$ 可满足, 则存在 c 的取值使 $F1$ 可满足。 4. 说明 equisatisfiable 和 equivalent 的区别。 5. 说明该证明在 Tseitin 转换中的作用。

B 按考试标准重写答案

一、三种敏感性与符号状态答案

flow-sensitive:

分析结果区分程序点，考虑语句执行顺序。
同一变量在不同程序点可以有不同抽象信息。

path-sensitive:

分析结果区分不同控制流路径。
同一程序点如果由不同路径到达，可以保留不同路径条件和不同状态。
符号执行是典型的路径敏感分析。

context-sensitive:

过程间分析时区分不同调用上下文。
同一函数从不同调用点、不同调用链进入时，可以得到不同分析结果。

程序分析结果:

流不敏感:
忽略两条赋值的先后关系，把 x 的可能值合并为 $\{0, 1\}$ ，所以 $y \in \{0, 1\}$ 。

流敏感但路径不敏感:
分析知道 $x=0$ 先执行，再经过条件分支；在合流点把两条路径合并，仍得到 $y \in \{0, 1\}$ 。

路径敏感:
路径 $c=true$: $x=1, y=1$ 。
路径 $c=false$: $x=0, y=0$ 。
两组状态分别保留，不立即合并。

调用上下文结果:

上下文不敏感:
把 $C1$ 、 $C2$ 两次调用合并分析， id 的参数/返回值集合为 $\{1, 2\}$ ，因此 a 、 b 都被近似为 $\{1, 2\}$ 。

上下文敏感:
区分 $C1$ 、 $C2$ ，得到 $a=1$ 、 $b=2$ 。

二、符号执行路径枚举答案

设输入符号为:

$\text{sigma0}(x) = \alpha$

路径一: 进入 then 分支:

条件: $\alpha > 0$
执行: $y = x + 1$
状态: $\text{sigma}(y) = \alpha + 1$

若进入 error:
路径条件 $\text{pi} = \alpha > 0 \text{ and } \alpha + 1 == 2$
化简得到 $\alpha = 1$ 。
该路径可满足。

路径二: 进入 else 分支:

条件: $\alpha \leq 0$
执行: $y = 1 - x$
状态: $\text{sigma}(y) = 1 - \alpha$
若进入 error:

路径条件 $\pi = \alpha \leq 0 \text{ and } 1 - \alpha = 2$
化简得到 $\alpha = -1$ 。
该路径可满足。

路径枚举表：

路径	路径条件	error 条件	可满足输入
then	$\alpha > 0$	$\alpha + 1 = 2$	$\alpha = 1$
else	$\alpha \leq 0$	$1 - \alpha = 2$	$\alpha = -1$

error() 可达：

可达。
输入 $x = 1$ 走 then 分支并使 $y = 2$ 。
输入 $x = -1$ 走 else 分支并使 $y = 2$ 。

路径爆炸：

每个条件分支都可能把路径数乘 2。
循环和递归还会产生大量甚至无限路径。
因此符号执行通常需要路径裁剪、循环界限、状态合并或约束求解优化。

三、SAT、Validity 与 CNF 答案

satisfiable:

公式可满足，表示存在一个解释或赋值，使公式取真。

valid:

公式有效，表示所有解释或赋值下公式都取真。

对偶关系：

公式 F valid, 当且仅当 $\text{not } F$ unsatisfiable。
公式 F satisfiable, 当且仅当 $\text{not } F$ not valid。

CNF:

CNF 是合取范式。
它是若干子句的合取，每个子句是若干文字的析取。
文字是变量或变量的否定。

判断：

$(a \text{ or } b) \text{ and } (\text{not } c \text{ or } d)$
是 CNF。它是两个析取子句的合取。

$\text{not } (a \text{ and } b)$
不是直接的 CNF，因为否定作用在复合公式上。
可等价改写为 $\text{not } a \text{ or } \text{not } b$ ，改写后是一个 CNF 子句。

$(a \text{ or } (b \text{ and } c))$
不是 CNF，因为析取内部嵌套了合取。
可分配为 $(a \text{ or } b) \text{ and } (a \text{ or } c)$ 。

四、Tseitin Transformation 答案

引入变量：

$x_1 \leftrightarrow (a \text{ or } b)$
 $x_2 \leftrightarrow (c \text{ and } d)$
 $x_3 \leftrightarrow \text{not } x_2$

$x4 \leftrightarrow (x1 \text{ and } x3)$
最终要求 $x4$ 为真。

Tseitin 变量表:

新变量	子公式
$x1$	$a \text{ or } b$
$x2$	$c \text{ and } d$
$x3$	$\text{not } x2$
$x4$	$x1 \text{ and } x3$

等价约束转 CNF:

$x1 \leftrightarrow (a \text{ or } b)$:
 $(\text{not } x1 \text{ or } a \text{ or } b)$
 $(x1 \text{ or not } a)$
 $(x1 \text{ or not } b)$

$x2 \leftrightarrow (c \text{ and } d)$:
 $(\text{not } x2 \text{ or } c)$
 $(\text{not } x2 \text{ or } d)$
 $(x2 \text{ or not } c \text{ or not } d)$

$x3 \leftrightarrow \text{not } x2$:
 $(\text{not } x3 \text{ or not } x2)$
 $(x3 \text{ or } x2)$

$x4 \leftrightarrow (x1 \text{ and } x3)$:
 $(\text{not } x4 \text{ or } x1)$
 $(\text{not } x4 \text{ or } x3)$
 $(x4 \text{ or not } x1 \text{ or not } x3)$

要求 $x4$:
 $(x4)$

最终 equisatisfiable CNF:

$(\text{not } x1 \text{ or } a \text{ or } b)$
 $\text{and } (x1 \text{ or not } a)$
 $\text{and } (x1 \text{ or not } b)$
 $\text{and } (\text{not } x2 \text{ or } c)$
 $\text{and } (\text{not } x2 \text{ or } d)$
 $\text{and } (x2 \text{ or not } c \text{ or not } d)$
 $\text{and } (\text{not } x3 \text{ or not } x2)$
 $\text{and } (x3 \text{ or } x2)$
 $\text{and } (\text{not } x4 \text{ or } x1)$
 $\text{and } (\text{not } x4 \text{ or } x3)$
 $\text{and } (x4 \text{ or not } x1 \text{ or not } x3)$
 $\text{and } (x4)$

优势:

直接等价展开到 CNF 可能指数膨胀。
Tseitin 转换为子公式引入新变量, 只要求可满足性等价, 不要求完全等价。
因此 CNF 大小通常线性增长, 更适合 SAT 求解器。

五、Equisatisfiability 证明答案

重写:

$A = a0 \text{ or } a1 \text{ or } \dots \text{ or } an$
 $B = b0 \text{ or } b1 \text{ or } \dots \text{ or } bm$
 $F1 = (A \text{ or } c) \text{ and } (B \text{ or not } c)$
 $F2 = A \text{ or } B$

$F1 \rightarrow F2$:

若 $F1$ 可满足, 则在某个赋值下 $(A \text{ or } c)$ 为真且 $(B \text{ or not } c)$ 为真。

如果 c 为真, 则第二个子句要求 B 为真, 因此 $A \text{ or } B$ 为真。

如果 c 为假, 则第一个子句要求 A 为真, 因此 $A \text{ or } B$ 为真。

所以 $F2$ 可满足。

$F2 \rightarrow F1$:

若 $F2$ 可满足, 则在某个赋值下 $A \text{ or } B$ 为真。

如果 A 为真, 令 $c = \text{false}$ 。

则 $A \text{ or } c$ 为真, $B \text{ or not } c$ 也因为 $\text{not } c$ 为真而为真。

如果 A 为假, 则 B 必为真, 令 $c = \text{true}$ 。

则 $A \text{ or } c$ 为真, $B \text{ or not } c$ 也因为 B 为真而为真。

因此存在 c 的取值使 $F1$ 可满足。

equisatisfiable 与 equivalent:

equisatisfiable 只要求两个公式同时可满足或同时不可满足。

equivalent 要求在所有赋值下真值完全相同。

Tseitin 转换通常只保持可满足性等价, 不保持原变量和新变量扩展空间上的完全等价。

作用:

Tseitin 转换把复杂公式转为 CNF 后交给 SAT 求解器。

SAT 求解只关心是否存在满足赋值, 因此保持 **equisatisfiability** 就足够。

这也是引入新变量但仍能正确判断原公式可满足性的原因。